

DRS High-Precision Distributed Synchronization

Vollständige Projektdokumentation

Autoren	Max Messavilla, Philipp Kodritsch
Datum	2026-05-25
Status	Implementiert und multi-node verifiziert (philipp-knode + drs-node-03)
Repository	<code>rpi_zeitsynchronisation/drs_sync/</code>
Plattform	Raspberry Pi 4B, PREEMPT_RT Linux
Sprache	C11 (Daemon), Python 3 (Agent), Vanilla JS (Web-UI)

Diese Dokumentation beschreibt ein deterministisches Userspace-Synchronisations-system für einen Cluster von Raspberry-Pi-Knoten ohne externe Zeitquelle.

Ziel: physische Synchronisationsabweichung $\Delta t_{\text{phys}} < 100 \mu\text{s}$ zwischen GPIO-18-Sync-Pulsen verschiedener Nodes.

Inhaltsverzeichnis

1	Mission und Übersicht	4
1.1	Mission Statement	4
1.2	Design-Philosophie	4
1.3	Synchronisations-Paradigma	4
1.4	Was das Projekt nicht ist	5
2	Anforderungen	5
2.1	Funktionale Anforderungen (32)	5
2.2	Nicht-funktionale Anforderungen (16)	6
2.3	Architektur-Verbote (eingehalten)	7
3	System-Architektur	7
3.1	Gesamt-Layout	7
3.2	CPU-Layout	8
3.3	Datenfluss in der Synchronisation	8
3.4	Hauptloop-Struktur	8
4	Module im Detail	9
4.1	Virtuelle Uhr (vclock)	9
4.2	Wire-Protokoll (proto_v2 + CRC32)	10
4.3	Netzwerk-I/O (net_io)	11
4.4	Min-Delay-Filter	12
4.5	Calibrate (Loopback-Latenz)	12
4.6	PI-Regler (Closed-Loop)	13
4.7	Leader-Election (sticky, term-basiert)	14
4.8	State Machine	14
4.9	GPIO + Pulse Engine	15
4.10	Telemetrie	16
4.11	Strukturiertes Error-Handling	17
4.12	Command-Channel	17
4.13	Lock-Mode (manuelle Rollenwahl)	18
5	Mathematische Grundlagen	18
5.1	Offset-Schätzung	18
5.2	Round-Trip-Delay	19
5.3	PI-Regler in Festkomma (positional)	19
5.4	Virtual-Clock-Mapping	19

6	Web-UI	20
6.1	Tab-Struktur	20
6.2	Sync-Tab im Detail	20
6.3	Polling-Strategie	21
6.4	Sicherheits-Modell	21
6.5	Peer-Auto-Discovery	21
6.6	WiFi-Transport und exklusiver Transport-Switch	22
7	HTTP-API-Referenz	22
7.1	GET /api/sync/state	22
7.2	GET /api/sync/health	23
7.3	POST /api/sync/control	23
7.4	GET /api/sync/cluster	23
7.5	GET /api/sync/errors?limit=N	23
7.6	Peer-, WiFi- und Transport-Endpoints	23
8	CLI-Tools	24
8.1	drs_sync_inspect	24
8.2	verify_pulse.py	24
9	Build-System	24
10	Deployment	25
10.1	Voraussetzungen	25
10.2	Erst-Provisionierung	26
10.3	Code-Update (laufender Pi)	26
10.4	Cluster-Deployment	27
11	Hardware-Setup	27
11.1	Pi 4B-Konfiguration	27
11.2	Netzwerk	27
11.3	GPIO-Verkabelung für externe Verifikation	28
12	Verifikation und Test	28
12.1	Acceptance-Kriterien (NF-1)	28
12.2	Test-Szenarien	28
12.3	Smoketest nach Deploy	29
12.4	Aktuelle Verifikationsstatus	29

13 Troubleshooting & FAQ	30
13.1 SSH-Probleme	30
13.2 Daemon startet, aber pulse_count bleibt 0	30
13.3 lat_corr ist absurd groß (>1 s)	30
13.4 sudo: unable to resolve host	31
13.5 State bleibt CANDIDATE / kommt nie zu LEADER	31
13.6 Web-UI: Cluster-Tabelle zeigt "unreachable"	31
14 Performance-Budget	31
15 Erweiterungs-Guide	32
15.1 Warum single-threaded?	32
15.2 Wie erweitern ohne Hot-Path zu stören?	32
15.3 Neuen Test hinzufügen	32
15.4 Neuen API-Endpoint hinzufügen	32
16 Glossar	33
17 Anhang: Dateipfade und Code-Referenzen	33
17.1 Verzeichnisstruktur	33
17.2 Wichtige Konstanten	34
17.3 Sub-Dokumentationen	35

1. Mission und Übersicht

1.1 Mission Statement

Das DRS-Projekt etabliert eine **deterministische, hochpräzise verteilte globale Zeitbasis** über einen dynamischen Cluster von Raspberry-Pi-Nodes — ausschließlich im **Linux-User-Space**, ohne Kernel-Module, ohne externe Zeitautoritäten (NTP/PTP/GPS).

Hartes Ziel: Physische Synchronisationsabweichung

$$\Delta t_{\text{physical}} < 100 \mu\text{s}$$

zwischen den Sync-Pulsen auf GPIO 18 verschiedener Nodes, gemessen extern via Logic-Analyzer oder Oszilloskop.

1.2 Design-Philosophie

KISS (Keep It Simple, Stupid). Das Protokoll vermeidet bewusst:

- Distributed-Consensus-Frameworks (kein Paxos, kein Raft)
- Dynamische Spanning-Trees / Routing-Protokolle
- Kernel-Modifikationen
- Externe Zeitautoritäten (NTP, GPS, PTP-Grandmaster)
- Schwergewichtige Serialisierung (kein Protobuf, kein JSON auf dem Draht)
- Multi-Hop-Routing
- Kryptographie im Sync-Hot-Path

Das System ist ausgelegt für:

- **Single-Hop Layer-2 Gigabit-Ethernet**
- **Trusted-Lab-Umgebung** (keine Auth, keine Crypto)
- **Fail-stop-Verhalten** der Nodes (keine byzantinischen Fehler)
- **Deterministische LAN-Bedingungen** für Mikrosekunden-Präzision (WLAN-Sync unterstützt, jedoch nur ms-genau; kein Internet)

1.3 Synchronisations-Paradigma

Internal Monotonic Synchronization. Ein einziger durch sticky, term-basierte Election bestimmter Leader (§4.7) definiert die Cluster-Referenz-Zeitlinie. Follower schätzen relativ zum Leader:

- **Phase-Offset** θ
- **Frequenz-Drift** (Rate)
- **Path-Delay** δ

Die Cluster-Zeitlinie basiert auf `CLOCK_MONOTONIC_RAW` und entspricht **nicht** UTC, NTP, GPS oder Wall-Clock-Zeit. Sie ist rein intern.

1.4 Was das Projekt nicht ist

- Kein NTP-Ersatz für UTC-Synchronisation
- Kein PTP (IEEE 1588) — nutzt keine HW-Timestamps; BCM54213PE im Pi 4B kann das ohnehin nicht
- Kein verteiltes Konsens-System für Daten (nur Zeit)

Transport-Hinweis: WLAN dient sowohl als Management-Kanal als auch als sekundärer Sync-Transport, allerdings in **Millisekunden-Klasse** — die NF-1-Garantie (<100 us) gilt **nur kabelgebunden** (Ethernet). eth0 und wlan0 dürfen nicht gleichzeitig im selben Subnetz aktiv sein (doppelte IP) → der Transport wird **exklusiv** umgeschaltet (siehe §3.4, §6, §7).

2. Anforderungen

2.1 Funktionale Anforderungen (32)

ID	Anforderung	Status / Modul
F-1	Autonomous multicast discovery	✓ <code>net_io.c</code> , 239.192.88.100:47200
F-2	Virtual global monotonic clock	✓ <code>vclock.{c,h}</code>
F-3	Min-delay filtering	✓ <code>min_delay.{c,h}</code> , Window=10
F-4	Deterministische Leader-Election	✓ <code>election.{c,h}</code> , sticky term-basiert (NodeID = Tiebreak)
F-5	Explicit fault recovery mapping	✓ <code>errors.{c,h}</code> + <code>ERRORS.md</code>
F-6	Lock-free telemetry	✓ <code>/dev/shm/drs-sync.state</code>
F-7	Stable convergence detection	✓ GPIO 23 nach 10-s-Stable
F-8	Immediate leader demotion	✓ <code>sm_on_announce()</code> , nur bei strikt höher-geremtem Leader
F-9	Non-blocking writer precedence	✓ Seqlock in <code>vclock.c</code>
F-10	Monotonic nanosecond representation	✓ <code>CLOCK_MONOTONIC_RAW</code>
F-11	GPIO validation pulse	✓ <code>pulse_engine</code> , GPIO 18 @ 1 Hz
F-12	Fixed performance governor	✓ <code>drs-perf-governor.service</code>
F-13	SO_TIMESTAMPING support	✓ Erfüllt durch Userspace-Timestamping
F-14	Automatic filter reset	✓ <code>min_delay_reset()</code> multiple Trigger
F-15	Fixed 64-byte packets	✓ <code>_Static_assert</code> Compile-Zeit
F-16	100 ms synchronization timeout	✓ Heartbeat + 3-miss-Hysterese
F-17	Step vs slew discipline	✓ <code>pi_ctrl.c</code> Dual-Loop
F-18	Fail-silent GPIO indicator	✓ GPIO 23 LOW nur bei Sync stable
F-19	Seamless late joiners	✓ FSM ohne Leader-Reset
F-20	Deterministic isolated startup	✓ GROUND 2 s, TX suppressed

ID	Anforderung	Status / Modul
F-21	Static latency calibration	✓ <code>calibrate.c</code> 50 Samples
F-22	Retry-storm suppression	✓ 5-Fail-Threshold
F-23	IRQ affinity isolation	✓ <code>drs-sync-irq-affinity.service</code>
F-24	Explicit endian-safe serialization	✓ <code>htonl/htons/htobe64</code>
F-25	Holdover freerun support	✓ Rate-Hold, Offset-Freeze
F-26	CRC packet validation	✓ IEEE 802.3 reflected
F-27	Sequence wraparound handling	✓ 16-bit unsigned semantics
F-28	Integral windup prevention	✓ Integrator-Clamp ± 1000 ppm
F-29	Step confirmation hysteresis	✓ 3 konsekutive Samples
F-30	Convergence signaling	✓ GPIO 23 + Telemetry-Feld
F-31	Automated calibration	✓ Trigger auto on startup
F-32	Calibration-on-election	✓ Trigger nach LEADER-Promote

Anmerkung zu F-13: Der Daemon verwendet Userspace-Timestamping, sodass alle Timestamps in derselben Clock-Domain liegen (`CLOCK_MONOTONIC_RAW`, in der auch die gesamte Synchronisations-Mathematik operiert). `Userspace-clock_gettime(CLOCK_MONOTONIC_RAW)` direkt vor `sendto` bzw. nach `recvfrom` liefert konsistent $\sim 3\text{--}4\mu\text{s}$ Loopback-Latenz und ist die korrekte Lösung für Pi 4B. Für PHY-basierte HW-Timestamping-Plattformen ist ein `SO_TIMESTAMPING`-Pfad denkbar.

2.2 Nicht-funktionale Anforderungen (16)

ID	Anforderung	Umsetzung
NF-1	Physical pulse delta $< 100\mu\text{s}$	Externe Verifikation via Logic Analyzer (siehe §12)
NF-2	Pure user-space implementation	Keine Kernel-Module, <code>/dev/gpiomem</code> , kein <code>adjtimex</code>
NF-3	Jitter resilience under mixed traffic	Min-Delay-Filter mit $10\mu\text{s}$ Toleranz
NF-4	Lock convergence within 10 s	Election + Calibration $\leq 5\text{ s}$, Sync $\leq 10\text{ s}$
NF-5	3-heartbeat hysteresis	Follower-Timeout = $3 \times 100\text{ ms}$
NF-6	Writer never blocked by readers	Seqlock-Pattern in <code>vclock</code> + Telemetry
NF-7	<code>SCHED_FIFO</code> priority 85	<code>rt_apply_all(cpu=3, prio=85)</code> + <code>systemd</code>
NF-8	<code>mlockall</code> memory locking	<code>rt_lock_memory()</code> + <code>LimitMEMLOCK=infinity</code>
NF-9	No hot-path disk or console I/O	Telemetrie via <code>/dev/shm-mmap</code> ; Errors via Ring
NF-10	Isolated Core 3 execution	<code>isolcpus=3 nohz_full=3 rcu_nocbs=3</code>
NF-11	Max slew = 1000 ppm	Rate-Clamp im PI-Controller
NF-12	No dynamic heap allocation in hot path	Pre-allocated Buffer vor <code>mlockall</code>
NF-13	No packet fragmentation	$64\text{ B} \ll \text{MTU } 1500$
NF-14	Deterministic packet parsing	Feste Offsets, kein struct-cast

ID	Anforderung	Umsetzung
NF-15	No mutex contention in sync loop	Seqlock + atomare 64-bit-Ops
NF-16	Core 3 sanctity	systemd CPUAffinity=3 + IRQ-Pinning

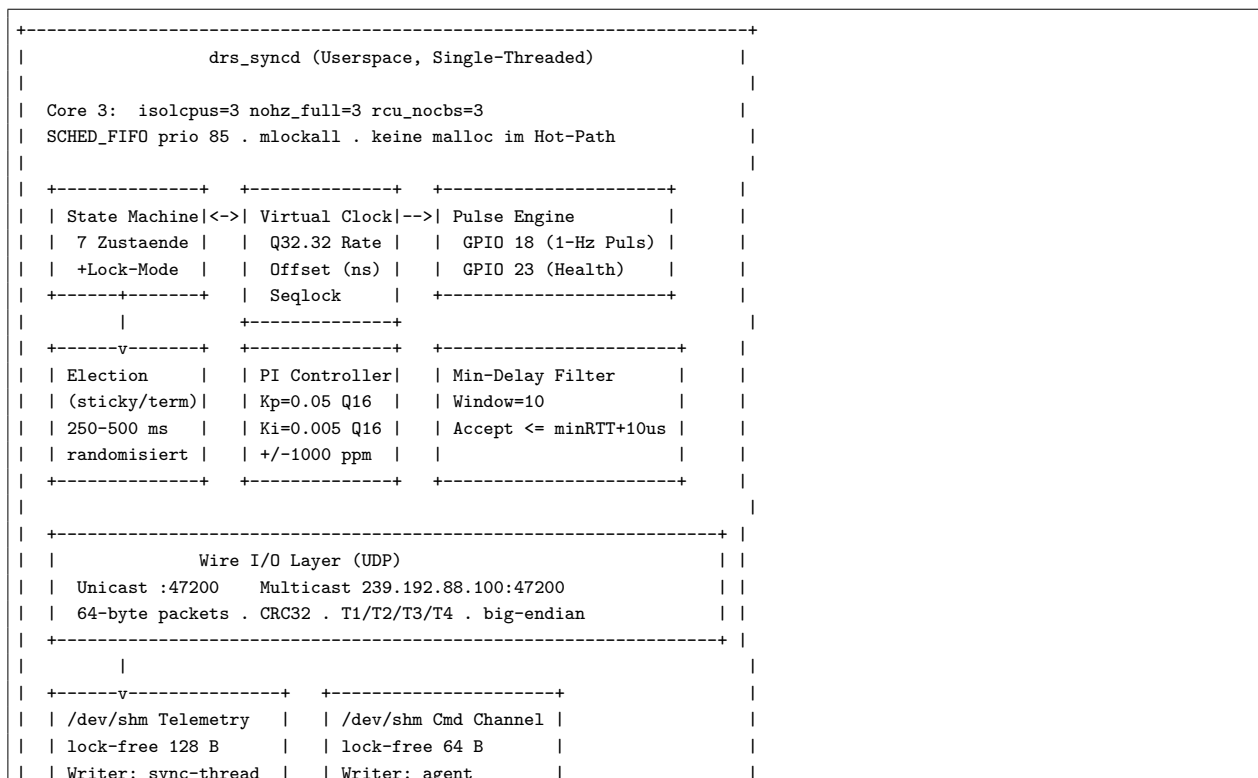
2.3 Architektur-Verbote (eingehalten)

- Kernel-Clocks modifizieren — vclock ist rein virtuell
- Custom Kernel-Module — nur Userspace + Standard-PREEMPT_RT-Kernel
- TCP — nur SOCK_DGRAM
- Distributed-Consensus — kein Quorum, keine Paxos-/Raft-Algorithmik (term-basierte sticky Leadership statt Quorum)
- Mutex im Hot-Path — Seqlock + atomare Operationen
- Filesystem-I/O im Sync-Loop — nur /dev/shm-mmap
- Float in RT-Thread — Q32.32 / Q16.16 Festkomma, __int128 für Zwischenergebnisse

Anmerkung: Sync ist interface-agnostisch (eth0 *oder* wlan0); IP_MULTICAST_IF wird auf die routbare DHCP-Adresse des aktiven Transports gebunden (§4.3). Die Sub-100- μ s-Garantie (NF-1) gilt nur auf dem kabelgebundenen Transport; WiFi-Sync ist ms-Klasse.

3. System-Architektur

3.1 Gesamt-Layout




```

| | Reader: agent/CLI | | Reader: sync-thread |
| +-----+ +-----+
+-----+-----+-----+
| | read-only | | write-only |
+-----+-----+-----+
| |
| | drs_agent (Python, Cores 0-2) |
| | HTTP API (Stdlib, ThreadPoolExecutor) |
| | Endpoints: /api/sync/{state, health, control, cluster, errors} |
| | Web-UI: /web/ (Vanilla JS, 2-s-Polling) |
+-----+-----+-----+

```

3.2 CPU-Layout

Core	Aufgabe	Quelle / Mechanismus
0-2	Linux-Default: Kernel, Userland, IRQs, drs_agent	systemd CPUAffinity=0-2 für Agent
3	Exklusiv für drs_syncd	isolcpus=3 + CPUAffinity=3 + IRQ-Pinning weg

CPU 3 sieht keinen Scheduler-Tick (`nohz_full=3`), keine RCU-Callbacks (`rcu_nocbs=3`), keine eth0/wlan-IRQs (`smp_affinity_list=0-2`). Die einzige Userspace-Aktivität auf Core 3 ist der `drs_syncd`-Prozess mit `SCHED_FIFO/85`.

3.3 Datenfluss in der Synchronisation

```

Follower-Node                                     Leader-Node
-----
T1 = CLOCK_MONOTONIC_RAW
sendto(SYNC_REQ) -----> recvfrom
                                T2 = CLOCK_MONOTONIC_RAW

                                T3 = CLOCK_MONOTONIC_RAW
recvfrom <----- sendto(SYNC_RESP)
T4 = CLOCK_MONOTONIC_RAW

theta = ((T2 - T1) + (T3 - T4)) / 2    <- Offset Follower vs Leader
delta = (T4 - T1) - (T3 - T2)         <- RTT

```

3.4 Hauptloop-Struktur

`drs_syncd` ist single-threaded (Begründung in §15.1) und blockiert in `epoll_wait()`. Die Eventquellen sind:

FD	Typ	Trigger
sock_ucast	UDP-Socket	Eingehende SYNC_REQ/RESP
sock_mcast	UDP-Socket	Eingehende ANNOUNCE
tick_fd	timerfd 50 ms	State-Machine-Tick, Telemetrie, Command-Polling
hb_fd	timerfd 100 ms	Leader: ANNOUNCE-Heartbeat senden
pe.timer_fd	timerfd one-shot ABS	Pulse-Rising-Edge auf GPIO 18
pe.pulse_low_fd	timerfd one-shot ABS	Pulse-Falling-Edge (10 ms nach Rising)

Jedes Event triggert eine kurze, blockierungsfreie Handler-Routine. Der längste Pfad ist die Pulse-Edge mit `gpio_write()` (Single 32-bit Store auf mmap'd Page $\rightarrow < 1 \mu s$).

Transport: Der Sync-Loop selbst ist interface-agnostisch — die Sockets binden an 0.0.0.0 und der TX-Multicast geht über `IP_MULTICAST_IF`, das auf die routbare Adresse des **aktiven** Transports

(eth0 oder wlan0) gesetzt wird (§4.3). Welches Interface aktiv ist, steuert der Agent über den exklusiven Transport-Switch (§6, §7); WiFi ist als Sync-Transport zulässig (ms-Klasse, NF-1 nur kabelgebunden).

4. Module im Detail

4.1 Virtuelle Uhr (vclock)

Datei: src/vclock.{c,h} (210 LOC).

Aufgabe: Eine virtuelle globale Zeitachse, unabhängig vom System-Clock. Sie bietet:

- Phase-Korrektur (offset_ns, signed int64)
- Frequenz-Skalierung (rate_q32_32, signed int64 in Q32.32)
- Statische Latenz-Korrektur (latency_corr_ns, signed int64)

Modell:

$$T_{\text{global}} = (T_{\text{local_raw}} \cdot \text{Rate}) + \text{Offset} - \text{LatencyCorrection}_{\text{ns}}$$

Die Implementation verwendet eine numerisch stabilere Variante mit Basis-Offset:

```
delta_local  = T_local_raw - base_local
delta_global = delta_local * Rate           // via __int128, shift >> 32
T_global     = base_global + delta_global + Offset - LatencyCorrection
```

Lock-Freiheit (Seqlock)

```
// Writer (drs_syncd Sync-Thread):
seq++;           // odd -> "writer active"
fence(release);
rate = new_rate; offset = new_offset; ...
fence(release);
seq++;           // even -> "consistent"

// Reader (agent, inspect, pulse_engine):
do {
    s1 = atomic_load(seq, acquire);
    if (s1 & 1) continue; // writer aktiv, retry
    rate = ...; offset = ...; ...
    s2 = atomic_load(seq, acquire);
} while (s1 != s2);
```

Festkomma mit __int128

- Rate-Multiplikation $T_{\text{local_delta}} \times \text{Rate}_{\text{Q32.32}}$ würde in 64 Bit überlaufen
- gcc/clang unterstützen __int128 auf aarch64 und x86_64 (compile-time `##error` sonst)
- Resultat-Shift `>>32` projiziert Q32.32 zurück auf Integer-ns

API:

- `vclock_init(vc, latency_corr_ns)` — Initial-Setup mit Rate=1.0, Offset=0
- `vclock_now(vc, t_local_raw_ns)` → T_{global} (Reader, lock-frei)
- `vclock_local_for_global(vc, t_global)` — Inverse, für Pulse-Timer
- `vclock_step_offset(vc, delta_ns, t_now)` — Hard-Step für $|\theta| > 1 \text{ ms}$
- `vclock_set_rate(vc, new_rate, t_now)` — Slew
- `vclock_snapshot(vc, &rate, &offset, &lat)` — Konsistenter Snapshot

Erfüllt: F-2, F-6, F-9, F-10, NF-6, NF-12, NF-15.

4.2 Wire-Protokoll (proto_v2 + CRC32)

Dateien: `src/proto_v2.{c,h}`, `src/crc32.{c,h}`.

Transport: UDP/IPv4, fixe 64 Byte pro Paket, Big-Endian, keine Fragmentierung.

Paket-Layout (64 Byte):

Off	Bytes	Feld	Bedeutung	Typ
0	4	magic	0x44525354 = "DRST"	u32 BE
4	1	version	0x02	u8
5	1	msg_type	1=ANNOUNCE, 2=SYNC_REQ, 3=SYNC_RESP	u8
6	1	flags	bit0=LEADER, bit1=HOLDOVER, bit2=CALIBRATED, bit3=FAULT	u8
7	1	reserved	= 0	u8
8	2	seq	Rolling	u16 BE
10	4	node_id	Eindeutig im Cluster	u32 BE
14	4	election_term	Monoton steigend	u32 BE
18	8	T1	Follower send time	u64 BE
26	8	T2	Leader receive time	u64 BE
34	8	T3	Leader send time	u64 BE
42	8	T4	Follower receive time	u64 BE
50	4	crc32	IEEE 802.3 reflected	u32 BE
54	10	padding	Zero, CRC-included	u8[10]
64				Total

Padding-Größe: Das Padding umfasst 10 Byte, sodass das Paket exakt 64 Byte gesamt ergibt (F-15, NF-13).

Serialisierung: Strikt field-by-field via `htonl/htons` und manuellem 64-bit-Big-Endian-Write (`proto_v2_serialize`).
Keine Struct-Casts (F-24).

CRC32:

- Polynom 0x04C11DB7 (IEEE 802.3), Init 0xFFFFFFFF, reflected I/O, XOR-out 0xFFFFFFFF
- 256-Eintrag-Lookup-Tabelle, einmalig in `crc32_init()` befüllt
- Bekannter Testvektor: `crc32("123456789") = 0xCBF43926`

Message-Types:

Code	Name	Verwendung	Pflichtfelder
0x01	ANNOUNCE	Leader → Multicast (100 ms Heart-beat)	node_id, election_term, T3
0x02	SYNC_REQ	Follower → Leader (Unicast, 50 ms)	T1, node_id
0x03	SYNC_RESP	Leader → Follower (Unicast)	T1 (echo), T2, T3

Erfüllt: F-15, F-24, F-26, NF-13, NF-14.

4.3 Netzwerk-I/O (net_io)

Datei: src/net_io.{c,h}.

Sockets:

Socket	Bind	Verwendung
sock_ucast	UDP 0.0.0.0:47200	SYNC_REQ + SYNC_RESP, auch TX-Multicast via IP_MULTICAST_IF
sock_mcast	UDP 0.0.0.0:47200 mit IP_ADD_MEMBERSHIP	ANNOUNCE-Empfang

Multicast-Setup:

- Group 239.192.88.100, Port 47200
- IP_ADD_MEMBERSHIP mit `imr_ifindex = if_nametoindex(<aktives Sync-Interface>)` (`eth0` oder `wlan0`)
- `IP_MULTICAST_LOOP = 0` (Sender bekommt eigene Pakete nicht zurück)
- `IP_MULTICAST_IF` = routbare IPv4 des aktiven Transports (`eth0` oder `wlan0`)

Adress-Präferenz (net_iface_pref_addr): Sowohl `IP_MULTICAST_IF` als auch die abgeleitete node-id nutzen die `eth0-/wlan0-IPv4` im **Subnetz des Default-Gateways** (routbare DHCP-Adresse, typ. 192.168.1.x), **nicht** die statische 10.0.0.x-Fallback-IP. `net_iface_pref_addr()` in `net_io.c` ermittelt das Gateway-Subnetz und wählt die passende Interface-Adresse; `node_id_from_ip()` in `drs_syncd.c` leitet die node-id aus dem **letzten Oktett der routbaren Adresse** ab. Die 10.0.0.XY-Adresse dient als DHCP-Down-Fallback (§11.2).

Socket-Options:

- `SO_REUSEADDR, SO_REUSEPORT` — schneller Restart ohne `TIME_WAIT`
- `IP_TOS = IPTOS_LOWDELAY (0x10)` — DSCP-Hint für QoS-fähige Switches
- `SOCK_NONBLOCK` — `recvmsg/sendto` blockieren nie

Timestamping-Entscheidung: Userspace `clock_gettime(CLOCK_MONOTONIC_RAW)` direkt vor `sendto` (T1, T3) und nach `recvfrom` (T2, T4). *Kein* `SO_TIMESTAMPING`-msg, da Kernel-SW-Timestamps in `CLOCK_REALTIME` zurückgegeben werden; Userspace-Timestamping hält alle Werte in einer einzigen Clock-Domain (`CLOCK_MONOTONIC_RAW`). Pi 4B's BCM54213PE PHY hat kein PTP-HW-Timestamping. Erwartete zusätzliche Jitter durch Userspace-TS: 5–10 μ s gegenüber Kernel-TS. Der Min-Delay-Filter entfernt diese Ausreißer.

4.4 Min-Delay-Filter

Datei: `src/min_delay.{c,h}`.

Aufgabe: Sample-Akzeptanz für PI-Controller-Updates. Verwirft Outlier (Scheduler-Spikes, IRQ-Stürme, Congestion-Bursts).

Algorithmus:

1. Rolling Ring-Buffer der letzten $N = 10$ RTT-Samples
2. `current_min = min(buffer)`
3. Sample-Akzeptanz: `rtt <= current_min + 10 \(\mu\)`s

Reset-Trigger (F-14):

- Leader-Change
- Sequence-Discontinuity (>1024 backward)
- Phase-Step >5 ms
- HOLDOVER-Entry
- LISTEN/CANDIDATE-Entry

Erfüllt: F-3, F-14, NF-3.

4.5 Calibrate (Loopback-Latenz)

Datei: `src/calibrate.{c,h}`.

Aufgabe: Statische Latenz-Korrektur ermitteln, die der Pi-spezifische Userspace+Kernel+NIC-Stack hat. Wird als `LatencyCorrection_ns` in die `vclock`-Formel eingerechnet.

Verfahren:

1. UDP-Socket-Paar (`send + recv`) auf `127.0.0.1:47201`
2. 50 Hin-und-Zurück-Zyklen, jeder mit `clock_gettime(CLOCK_MONOTONIC_RAW)` vor `sendto` und nach `recvfrom`
3. `mn = min(samples)` — bestmögliche Loopback-RTT
4. Outlier-Rejection: Samples $> mn + 20 \(\mu\)$ s werden verworfen
5. Rückgabewert: `mn / 2` (einseitige Latenz)

Erwartung auf Pi 4B: $\sim 3\text{--}4 \mu\text{s}$.

Trigger (F-31, F-32):

- Startup (Übergang GROUND $\rightarrow$ CALIBRATION $\rightarrow$ LISTEN)
- Nach Leader-Election (Calibration-on-Election)
- Nach HOLDOVER-Expiration
- Manuell via Command-Channel (`recalibrate`)

Erfüllt: F-21, F-31, F-32.

4.6 PI-Regler (Closed-Loop)

Datei: `src/pi_ctrl.{c,h}`.

Regelgröße = globaler Offset (nicht Roh- θ). Der Regler wird mit dem vClock-relativen Offset $e = \text{led_mid} - \text{vclock_now}(\text{fol_mid})$ gespeist (Leader-Uhr minus Follower-vClock). Im Gegensatz zum rohen θ (reiner `CLOCK_MONOTONIC_RAW`-Abstand, von der Rate unabhängig) reagiert e auf die Rate — die Schleife ist geschlossen und der Integrator konvergiert gegen den echten Frequenzfehler ($\sim 0,5$ ppm), ohne in den Anti-Windup-Clamp zu laufen.

Step-Mode (schnell, sprungartig):

- Trigger: 3 konsekutive Samples mit $|e| > 1$ ms (F-29, F-17)
- Aktion: `vclock_step_offset(+e, t_now)` (hebt die Follower-vClock auf den Leader); danach `Rate→Nominal, pi_ctrl_init, min_delay_reset`
- Ohne `step_accum`: `vclock_now()` spiegelt einen ausgeführten Step beim nächsten Sample bereits wider.

Slew-Mode (sanft, frequenzbasiert):

- Aktiv bei $|e| \leq 1$ ms
- **Positionale** Formel: $\text{Rate} = \text{Rate}_{\text{nominal}} + K_p \cdot e + K_i \cdot \int e \, dt$ (NICHT $\text{Rate}_{\text{old}} + \dots$)
- Festkomma: $K_p = 3276 \text{ Q16.16} \approx 0,05$, $K_i = 327 \text{ Q16.16} \approx 0,005$
- Update-Periode: 50 ms (synchron mit Sync-Exchange-Periode)

Anti-Windup (F-28): Integrator auf ± 1000 ppm geklemmt — im Closed-Loop dient der Clamp als Schutz und wird im Normalbetrieb nicht erreicht.

Rate-Clamp (NF-11): $\text{Rate} \in [\text{Nominal} - 1000 \text{ ppm}, \text{Nominal} + 1000 \text{ ppm}]$.

Discipline-Reset bei Leader-Wechsel: Der Follower merkt sich den disziplinierten Leader in `locked_leader_id`. Wechselt der amtierende Leader (Failover, Re-Election), wird der gesamte Discipline-State zurückgesetzt: `min_delay_reset` (neuer `min_delay`-Pfad, da die Pfadlatenz zu einem anderen Leader anders ist) *und* `pi_ctrl_init` (Integrator + Step-Hysterese auf 0). So regelt der Regler stets gegen den Frequenzfehler des aktuellen Leaders (§4.8 Failover-Re-Acquire).

Numerische Strategie:

```
// Q16.16; / 65536 (nicht >>16: vorzeichensymmetrisch) und x1e6 (ppm):
int64_t kp_term_ppm = (KP_Q16 * e_ns * 1000000LL / period_ns) / 65536;
int64_t ki_term_ppm = (KI_Q16 * e_ns * 1000000LL / period_ns) / 65536;
integrator = clamp(integrator + ki_term_ppm, -1000, 1000);
int64_t total_ppm = kp_term_ppm + integrator;
// 1 ppm = (2^32 / 1e6) ~ 4295 in Q32.32:
int64_t rate_delta_q32 = total_ppm * 4295;
new_rate = clamp(NOMINAL_Q32 + rate_delta_q32, MIN_Q32, MAX_Q32); // positional
```

Keine Floats: Architecture-Constraint. RT-Threads dürfen keine Float-Exceptions auslösen.

Erfüllt: F-17, F-28, F-29, F-33, F-39, NF-11.

4.7 Leader-Election (sticky, term-basiert)

Datei: `src/election.{c,h}`.

Algorithmus: Sticky term-basierte Leadership.

Kern-Prinzip: Der **amtierende Leader bleibt Leader, bis er ausfällt**. Maßgeblich ist der höchste **Election-Term**, nicht die niedrigste NodeID. Ein zurückkehrender Knoten — frisch neu-gestartet (Term 0) oder mit gezogenem Kabel und veraltetem Term — wird **FOLLOWER** und reißt die Führung **nicht** an sich. So entsteht bei einem Node-Restart kein unnötiger Leader-Flap.

Regeln:

- Sticky Leadership: höchster Election-Term gewinnt. Der amtierende Leader demotet **nur** bei einem **strikt höher-getermten** Leader.
- NodeID = reiner Tiebreak bei gleicher Term-Nummer in einer echten Election (z. B. gleichzeitiger Kaltstart). `node_id` = letztes Oktett der routbaren IPv4 (§4.3)
- Election-Timeout randomisiert in [250 ms, 500 ms] — vermeidet Kollisionen
- ElectionTerm monoton steigend pro echter Election-Runde

Beobachtungs-Logik:

```
if (peer_term > our_term)      -> uebernehmen (strikt hoeher), become_follower=true
if (peer_term < our_term)     -> ignorieren (wir sind frischer; Leader bleibt)
if (peer_term == our_term):
    if (peer_id < own_id)      -> become_follower=true (Tiebreak)
    else                      -> ignorieren
```

Ein neu gestarteter Knoten mit niedriger NodeID, aber **niedrigerem Term**, fällt in den `peer_term < our_term`-Zweig und wird FOLLOWER — er übernimmt **nicht** die Führung vom amtierenden Leader. Die NodeID entscheidet ausschließlich als Tiebreak bei gleichem Term.

Random-Source: Einmaliger Seed aus `/dev/urandom`, danach xorshift64.

Erfüllt: F-4, F-8, F-19.

4.8 State Machine

Dateien: `src/statemachine.{c,h}`.

Zustände (7):

Zustand	Beschreibung
GROUND	Startup-Stabilisierung, 2 s, Filter zeroed, TX suppressed
CALIBRATION	Self-Loopback-Latenz-Messung (50 Samples)
LISTEN	Passive Discovery, Election-Timeout läuft
CANDIDATE	Election in Progress
FOLLOWER	Synchronisiert zu Leader, SYNC_REQ alle 50 ms
LEADER	Authoritative Clock Source, ANNOUNCE alle 100 ms
HOLDOVER	Temporärer Freerun (Rate eingefroren, GPIO 23 HIGH)

Transitionen:

Von	Trigger		Nach	Aktion
GROUND	2 s elapsed		CALIBRATION	Start Self-Calibration
CALIBRATION	50 Samples ok		LISTEN	vclock_init, Pulse-Engine aktivieren, Timeout armen
LISTEN	ANNOUNCE (sm_is_active)	empfangen	FOLLOWER	Sync-Request beginnen
LISTEN	Election-Timeout		CANDIDATE	Timeout re-armen
CANDIDATE	ANNOUNCE höher-getermt Tiebreak	/	FOLLOWER	leader_id übernehmen
CANDIDATE	Election-Timeout		LEADER	term++, Heartbeats starten
LEADER	ANNOUNCE strikt höherer Term		CANDIDATE	Demotion (nur bei höherem Term)
FOLLOWER	3 Heartbeats verpasst		HOLDOVER	Rate-Freeze, GPIO 23 HIGH, pending SYNC_REQ löschen
HOLDOVER	ANNOUNCE empfangen		FOLLOWER	Re-Sync (Discipline-Reset bei neuem Leader)
HOLDOVER	10 s elapsed		CANDIDATE	Re-Election, Filter-Reset

Active-before-Follower (sm_is_active-Guard): Ein Knoten muss den Pfad **GROUND** → **CALIBRATION** → **LISTEN** vollständig abgeschlossen haben — Loopback-Kalibrierung, **vclock_init** und **Pulse-Engine aktiviert** — **bevor** er FOLLOWER werden darf. Ein direkter GROUND→FOLLOWER-Sprung auf ein frühes ANNOUNCE ist gesperrt (sm_is_active-Guard in sm_on_announce). So ist bei Eintritt in FOLLOWER stets eine gültige Latenz-Korrektur vorhanden und die Pulse-Engine aktiv.

Failover-Re-Acquire: Der ausstehende SYNC_REQ ("pending", auf eine SYNC_RESP wartend) hat ein Timeout: er läuft nach DRS_SYNC_REQ_TIMEOUT_NS ab und wird beim Verlassen von FOLLOWER (Richtung HOLDOVER/Re-Election) gelöscht. So synchronisiert der Follower nach einem Leader-Disconnect sauber auf einen neuen Leader neu. Bei einem Leader-Wechsel wird zusätzlich die Discipline (min_delay + PI) zurückgesetzt (locked_leader_id, siehe §4.6).

Lock-Mode-Override (siehe §4.13):

- LOCK_AS_LEADER: Fremde ANNOUNCEs in sm_on_announce direkt verworfen
- LOCK_AS_FOLLOWER: In LISTEN bleibt der Node trotz Timeout passiv

Erfüllt: F-1, F-4, F-7, F-8, F-19, F-20, F-25, NF-4, NF-5.

4.9 GPIO + Pulse Engine

Dateien: src/gpio_bcm2711.{c,h}, src/pulse_engine.{c,h}.

Hardware-Zugriff:

- mmap auf /dev/gpiomem (kein /dev/mem, ohne CAP_SYS_RAWIO)
- BCM2711-Register-Direktzugriff:
 - GPFSEL0..2 (Offsets 0/4/8) — Funktion-Select 3 Bit/Pin
 - GPSET0 (Offset 0x1C) — write-1 → Pin HIGH
 - GPCLR0 (Offset 0x28) — write-1 → Pin LOW

- Latenz pro `gpio_write()`: 32-bit-Store auf `mmap`'d Page → typisch $< 1 \mu\text{s}$

Pulse Engine:

- GPIO 18 = Sync-Puls: 10 ms HIGH ab globaler Sekunde, dann LOW
- GPIO 23 = Health-Indikator: LOW nur wenn $|\text{offset}| < 100 \mu\text{s}$ für 10 s am Stück

Pulse-Scheduling-Logik:

```
1. T_now_global = vclock_now(vc, t_local_raw)
2. T_next_global_sec = ceil((T_now_global + 1ms) / 1e9) * 1e9
3. T_next_local = vclock_local_for_global(vc, T_next_global_sec)
4. timerfd_settime(pe.timer_fd, TFD_TIMER_ABSTIME, T_next_local)
```

Wichtig: `arm_next()` wird nur nach LISTEN-Transition und nach jedem TAG_PULSE-Event aufgerufen — nicht im 50-ms-Tick, da ein Re-Arm im Tick den Push-Forward-Check triggern und den Timer ausführungsfähig halten würde.

Erfüllt: F-11, F-18, F-30, NF-1.

4.10 Telemetrie

Datei: `src/telemetry.{c,h}`, `/dev/shm/drs-sync.state` (128 Byte).

Layout (Reihenfolge entscheidend für `agent.py struct.unpack()`):

```
struct drs_telemetry {
    uint32_t pid;
    uint32_t version;
    volatile uint32_t state;           // drs_state_t
    volatile uint32_t flags;          // bitmask
    volatile uint32_t leader_node_id;
    volatile uint32_t election_term;
    volatile int64_t offset_ns;
    volatile int64_t rate_q32_32;
    volatile int64_t latency_corr_ns;
    volatile uint64_t last_rtt_ns;
    volatile uint64_t convergence_unix_ns;
    volatile uint32_t holdover_remaining_ms;
    volatile uint32_t error_code_last;
    volatile uint32_t error_count;
    volatile uint32_t pulse_count;
    volatile uint32_t lock_mode;      // drs_lock_mode_t
    uint8_t reserved[36];
};
```

Konsistenz:

- Writer: `drs_syncd-Sync-Thread` (Single-Writer)
- Reader: `agent.py`, `drs_sync_inspect` (Multi-Reader, read-only `mmap`)
- Atomare 64-bit-Writes für `offset_ns/rate_q32_32` — natürlich aligned
- Kein Lock nötig (NF-6, NF-9, NF-15)

Update-Frequenz: Bei jedem 50-ms-Tick im Sync-Daemon.

Erfüllt: F-6, NF-6, NF-9.

4.11 Strukturiertes Error-Handling

Datei: `src/errors.{c,h}`, `/dev/shm/drs-sync.errors` (4 KB Ring-Buffer).

Designprinzip: Hot-Path schreibt nur in atomic Ring-Buffer. Logger-Thread würde asynchron drainieren (außerhalb Hot-Path).

Error-Codes:

Code	Name	Ursache	Aktion	Indikation
1	CALIB_TIMEOUT	Loopback keine Samples	Re-calibrate (max 3×)	GPIO 23 HIGH
2	HOLDOVER_EXPIRED	10s ohne Heartbeat	Re-Election	GPIO 23 HIGH
3	CRC_MISMATCH	Eingehendes Paket korrupt	Verwerfen + Counter++	Telemetry only
4	SEQ_DISCONTINUITY	Seq-Sprung >1024	Filter-Reset	GPIO 23 HIGH
5	GPIO_OPEN_FAIL	/dev/gpiomem nicht zugänglich	Exit 70	systemd-Restart
6	RT_PRIO_FAIL	sched_setscheduler fail	Warnung, weiter	stderr
7	TX_TIMESTAMP_LOST	MSG_ERRQUEUE leer	Sample verwerfen	Counter
8	RX_TIMEOUT	epoll_wait fail	Exit	journal
9	INVALID_PACKET	Magic/Version/Größe falsch	Verwerfen	Counter
10	NET_INIT_FAIL	net_io_init fail	Exit 70	—
11	SHM_INIT_FAIL	/dev/shm nicht beschreibbar	Telemetry-off, Sync läuft	stderr

Probe-Effekt-Disziplin: Sync-Hot-Path ruft niemals `printf/fprintf/syslog`. Atomic Ring-Buffer-Slot-Write ist O(1) ohne Lock. Agent (oder Logger-Thread) drainiert asynchron.

Erfüllt: F-5, NF-9.

4.12 Command-Channel

Datei: `src/cmd.{c,h}`, `/dev/shm/drs-sync.cmd` (64 Byte).

Designziel: Externe Steuerung (vom Agent oder CLI) ohne den Sync-Thread zu unterbrechen.

```
struct drs_cmd_channel {
    volatile uint32_t seq;           // atomic monotone
    volatile uint32_t code;         // drs_cmd_code_t
    volatile uint64_t unix_ns;      // Diagnostic timestamp
    uint8_t payload[48];           // Reserviert
};
```

Polling-Pattern:

```
// Im Daemon-Tick (alle 50 ms):
uint32_t current = atomic_load(&cmdc->seq, acquire);
if (current != last_seen_seq) {
    code = cmdc->code;
    last_seen_seq = current;
    // dispatch by code
}
```

Command-Codes:

Code	Name	Aktion
0	NONE	(sentinel)
1	RECALIBRATE	calibrate_loopback() + vclock_init + Filter-Reset
2	FORCE_HOLDOVER	State → HOLDOVER, Offset-Freeze, GPIO 23 HIGH
3	FORCE_DEMOTE	LEADER → CANDIDATE
4	RESET_FILTERS	min_delay_reset + pi_ctrl_init
5	FORCE_LEADER	Lock als LEADER (term + 1000)
6	FORCE_FOLLOWER	Lock als FOLLOWER (HOLDOVER if was LEADER)
7	AUTO_MODE	Lock-Mode zurück auf AUTO

4.13 Lock-Mode (manuelle Rollenwahl)

Motivation: Für Test-Szenario 3 (Crash Failure Tolerance) und manuelle Failover-Tests muss ein Node deterministisch als LEADER bzw. FOLLOWER festgesetzt werden können. Die automatische Election allein gibt diese Kontrolle nicht.

Drei Modi:

```
typedef enum {
    DRS_LOCK_AUTO          = 0,    // normale term-basierte Election
    DRS_LOCK_AS_LEADER     = 1,    // Lock als LEADER
    DRS_LOCK_AS_FOLLOWER   = 2,    // niemals LEADER werden
} drs_lock_mode_t;
```

Effekte:

Mode	sm_on_announce	sm_on_tick	Bei Mode-Wechsel
AUTO	Normal	Normal	—
LOCK_AS_LEADER	Fremde ANNOUN- CEs ignoriert	—	term + 1000, state → LEADER
LOCK_AS_FOLLOWER	Normal (kann Fol- lower werden)	Election-Timeout ignoriert	LEADER → HOL- DOVER, CANDIDA- TE → LISTEN

End-to-End-Test (verifiziert auf philippknode):

```
AUTO          -> state=LEADER, term=1    (normale term-basierte Election)
force-leader  -> state=LEADER, term=1001  (Boost um sicher zu sein)
force-follower -> state=HOLDOVER          (war LEADER -> demoted)
auto-mode     -> state=HOLDOVER -> (10 s) -> CANDIDATE -> LEADER, term=1002
```

5. Mathematische Grundlagen

5.1 Offset-Schätzung

Klassische PTP-/NTP-Zwei-Wege-Messung:

$$\theta = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}$$

mit

- T_1 — Follower-Sendezeit (lokal)

- T_2 — Leader-Empfangszeit (remote)
- T_3 — Leader-Sendezeit (remote)
- T_4 — Follower-Empfangszeit (lokal)

Annahme: Path-Delay ist *symmetrisch*. Asymmetrie (z. B. Switch-Queue-Längen) erzeugt residuellen Offset, den der Min-Delay-Filter minimiert.

5.2 Round-Trip-Delay

$$\delta = (T_4 - T_1) - (T_3 - T_2)$$

- $T_4 - T_1$ = Gesamt-Round-Trip am Follower
- $T_3 - T_2$ = Verarbeitungszeit beim Leader
- Differenz = reine Netzwerk-RTT

5.3 PI-Regler in Festkomma (positional)

Regelgröße ist der globale Offset $e = \text{led_mid} - \text{vclock_now}(\text{fol_mid})$, nicht das rohe θ .

Slew-Update (jede 50 ms wenn $|e| \leq 1 \text{ ms}$):

$$P_{\text{term}} = K_p \cdot e / \text{period} \quad (1)$$

$$I_{\text{term}} = \text{clamp}(I_{\text{term}} + K_i \cdot e / \text{period}, \pm 1000 \text{ ppm}) \quad (2)$$

$$\text{total_ppm} = P_{\text{term}} + I_{\text{term}} \quad (3)$$

$$\text{rate_delta}_{Q32} = \text{total_ppm} \cdot \frac{2^{32}}{10^6} \quad (4)$$

$$\text{rate_new} = \text{clamp}(\text{NOMINAL} + \text{rate_delta}_{Q32}, R_{\min}, R_{\max}) \quad (\text{positional}) \quad (5)$$

Konstanten:

$$K_p = 0,05 \approx 3276 \quad (\text{Q16.16}) \quad (6)$$

$$K_i = 0,005 \approx 327 \quad (\text{Q16.16}) \quad (7)$$

$$1 \text{ ppm} = \frac{2^{32}}{10^6} \approx 4295 \quad (\text{Q32.32 Rate-Tick}) \quad (8)$$

5.4 Virtual-Clock-Mapping

Vorwärts:

$$T_{\text{global}} = (T_{\text{local_raw}} - \text{base_local}) \cdot \text{Rate} \gg 32 + \text{base_global} + \text{Offset} - \text{LatencyCorrection}$$

base_local und base_global werden bei Rate-Change und Step-Offset gleichzeitig aktualisiert, damit T_{global} stetig bleibt (kein Knick).

Inverse (für Pulse-Timer-Scheduling):

$$y = T_{\text{global}} - \text{base_global} - \text{Offset} + \text{LatencyCorrection} \quad (9)$$

$$T_{\text{local_raw}} = \text{base_local} + \frac{y \ll 32}{\text{Rate}} \quad (10)$$

Beide Operationen nutzen `__int128` für Zwischenergebnisse.

6. Web-UI

Pfade: `web/index.html`, `web/app.js`, `web/style.css`.

URL: <http://192.168.1.202:8080/> bzw. <http://philippknode.lan:8080/> (mit mDNS/hosts).

6.1 Tab-Struktur

Tab	Zweck
Topology	Cytoscape-Graph, Subnet-Scan, Port-Discovery
Network	eth0-Konfiguration, Echo-Port
Measurement	RTT-Mess-Runs (Legacy)
Sync	Status, Pre-Flight-Checks, Steuerung, Cluster
System	Hostname, Kernel, Temperatur, Uptime

6.2 Sync-Tab im Detail

```
+-- A. Status -----+
| Daemon: . running (PID 4873) |
| State: * LEADER |
| Lock Mode: [AUTO] |
| Leader: node 201 (term 1) |
| Offset: 0 ns Rate: 0.00 ppm Pulses: 123 |
| Errors: 0 (last code 0) Holdover: -- |
+-----+

+-- B. Pre-Flight Checks 12/12 OK . 0 critical . 0 warning -----+
| + PREEMPT_RT Kernel Linux 6.12.75-rt ... |
| + CPU 3 isolated (isolcpus=3) /proc/cmdline OK |
| + nohz_full + rcu_nocbs on Core 3 |
| + CPU 3 governor = performance |
| + eth0 IRQs not on Core 3 |
| + GPIO access (/dev/gpiomem) |
| + Multicast routing |
| + eth0 link state |
| + No competing time services |
| + drs-sync.service active |
| + Telemetry channel readable |
| + Sync converged |offset|<100us |
+-----+

+-- C. Steuerung -----+
| Rolle: (.) Auto (term-basierte Election) |
| ( ) Force LEADER |
| ( ) Force FOLLOWER |
| Lifecycle: [Start] [Stop] [Restart] |
| Operations: [Recalibrate] [Reset Filters] |
| Tests: [Force Holdover] [Force Demote (Leader)] |
+-----+

+-- D. Cluster 1/2 reachable . leader: 201 . max |D|: 0 ns -----+
| Label IP State Leader Offset Errors |
| scanned 192.168.1.115 unreachable: ... |
| scanned 192.168.1.201 * LEADER 201 0 ns 0 |
| [Auto-discover peers] |
+-----+

+-- E. Sync transport -----+
| (.) Ethernet (eth0) ( ) WiFi (wlan0) [Apply] |
| Aktiv: eth0 . WiFi sticky: Fallback nur bei WLAN-Verlust |
+-----+
```

```

+- F. WiFi -----+
| Status: wlan0 connected . SSID "lab-ap" . -52 dBm |
| [Scan] SSID v PSK [.....] [Connect] |
+-----+

```

Cards

Auto-discover-Button (Cluster-Card): Löst `POST /api/peers/auto_discover` aus — der Agent scant das DHCP-Subnetz selbst und pflegt `peers.json` (§6.5).

Sync-transport-Selector (Card E): Radio-Buttons Ethernet/WiFi. "Apply" sendet `POST /api/transport {mode}`. **WiFi ist sticky** — kein Confirm/Timer: ein Konnektivitäts-Watchdog (`_transport_watchdog`, alle `DRS_WIFI_WATCHDOG_S = 15 s`) fällt erst nach `DRS_WIFI_FAIL_LIMIT = 3` aufeinanderfolgenden Fehlern (45 s echter WLAN-Verlust) auf `eth0` zurück. `eth0` und `wlan0` sind **exklusiv** (§6.6).

WiFi-Card (Card F): Scan/Connect/Status über einen reinen Python-wpa_supplicant-Control-Socket-Client (kein `nmcli/iw`; NetworkManager masked). `GET /api/wifi/scan`, `POST /api/wifi/connect {ssid,psk}`, `GET /api/wifi/status`.

6.3 Polling-Strategie

- **Status + letzte Errors:** alle 2 s (`/api/sync/state`, `/api/sync/errors?limit=5`)
- **Cluster:** jedes zweite Tick (~4 s) — Subprocess via `ThreadPoolExecutor` pro Peer
- **Health-Checks:** nur beim Tab-Öffnen + Click auf "re-run" (teure subprocess-Calls)
- **Action-Log:** `localStorage`, kein Backend-State

6.4 Sicherheits-Modell

`/api/sync/control` ist eine Privilege-Eskalation (`systemctl restart + /dev/shm/drs-sync.cmd-Write`). Das System ist explizit für **Trusted-Lab-Umgebungen** ausgelegt. Keine Auth, kein CSRF-Token, keine TLS. Allow-List der Actions hardkodiert in `execute_sync_control()`.

6.5 Peer-Auto-Discovery

Der Agent findet seine Peers selbst und ergänzt damit manuell gepflegte `peers.json`-Einträge.

Discovery-Mechanismus:

1. Bestimmt das eigene DHCP-Management-Subnetz (`_mgmt_subnet`)
2. Scant das Subnetz auf DRS-Echo (Port 5005) und macht einen `:8080-Sweep`
3. Probt `:8080/api/system` jedes Treffers für den Hostnamen
4. Aktualisiert `peers.json` automatisch: Feld `source = "auto"/"manual"`, Feld `cluster (bool)`, aktuelle DHCP-IP gepflegt, **self** ausgeschlossen, veraltete Auto-Einträge entfernt, manuelle Einträge gepinnt

Auslösung: `POST /api/peers/auto_discover`, ein Hintergrund-Loop sowie der Web-UI-Button. `aggregate_cluster` fragt ausschließlich **cluster-fähige** Peers ab.

6.6 WiFi-Transport und exklusiver Transport-Switch

WiFi Scan/Connect läuft über einen reinen Python-Client am wpa_supplicant-Control-Socket — bewusst **ohne** nmcli/iw, da NetworkManager masked ist. Endpoints: GET /api/wifi/scan, POST /api/wifi/connect {ssid,psk}, GET /api/wifi/status.

Country-Unlock: Das RPi-WLAN ist per rfkill gesperrt, bis ein Regulatory-Country gesetzt ist. Der Agent setzt es via raspi-config do_wifi_country DE plus sysfs-Unblock und lädt die Config per networkctl reload neu (**kein** networkd-Restart, der eth0 flappen würde).

Exklusiver eth0↔wlan0-Switch: WiFi und eth0 dürfen **nicht** gleichzeitig im selben Subnetz aktiv sein (doppelte IP). Der Transport wird daher exklusiv umgeschaltet über POST /api/transport {mode:eth|wifi}:

- eth0 wird erst gesenkt, **wenn** wlan0 eine IP hat
- **WiFi ist sticky:** ein Konnektivitäts-Watchdog (_transport_watchdog, alle DRS_WIFI_WATCHDOG_S = 15 s) prüft, ob wlan0 eine IP hat und das Default-Gateway pingbar ist, und fällt erst nach DRS_WIFI_FAIL_LIMIT = 3 Fehlern (45 s) auf eth0 zurück — kein Timer, kein Confirm
- **die Wahl überlebt den Reboot:** set_transport() schreibt den Modus nach STATE_DIR/transport; beim Agent-Start stellt _apply_persisted_transport() ihn wieder her (ein auf WiFi belassener Knoten kommt nach dem Neustart wieder auf WiFi hoch)
- das drs_syncd-Interface (drs_syncd-iface) wird über ein systemd-Drop-in gesetzt

7. HTTP-API-Referenz

Alle Endpunkte unter http://<host>:8080/api/sync/.... Content-Type application/json.

7.1 GET /api/sync/state

Aktueller Live-Zustand des lokalen drs_syncd.

Response:

```
{
  "pid": 4873,
  "version": 1,
  "state": "LEADER",
  "state_id": 5,
  "flags": 1,
  "is_leader": true,
  "leader_node_id": 201,
  "election_term": 1,
  "offset_ns": 0,
  "rate_ppm": 0.0,
  "latency_correction_ns": 3962,
  "last_rtt_ns": 0,
  "convergence_unix_ns": 0,
  "holdover_remaining_ms": 0,
  "error_code_last": 0,
  "error_count": 0,
  "pulse_count": 123,
  "lock_mode": "AUTO",
  "lock_mode_id": 0
}
```

Errors: 503 wenn /dev/shm/drs-sync.state nicht existiert.

7.2 GET /api/sync/health

Pre-Flight-Diagnose. Führt 12 Checks aus. Severity: **critical** (System unbrauchbar wenn fail), **warning** (System läuft aber Anforderung verletzt), **info** (Hinweis).

7.3 POST /api/sync/control

Steuerung des Daemons. Body {"action": "<action>"}

Action	Implementation	Effekt
start	systemctl	Daemon starten
stop	systemctl	Daemon stoppen
restart	systemctl	Reset, frischer GROUND-Durchlauf
recalibrate	cmd 1	Loopback-Calibration neu
force-holdover	cmd 2	State → HOLDOVER
force-demote	cmd 3	LEADER → CANDIDATE
reset-filters	cmd 4	min_delay + PI Reset
force-leader	cmd 5	Lock als LEADER
force-follower	cmd 6	Lock als FOLLOWER
auto-mode	cmd 7	Lock-Mode → AUTO

Response (Erfolg):

```
{"ok": true, "seq": 9, "code": 7, "action": "auto-mode"}
```

7.4 GET /api/sync/cluster

Aggregierte Sicht über alle in `peers.json` eingetragenen Peers. Agent fragt `/api/sync/state` aller Peers parallel via `ThreadPoolExecutor` (timeout 2s pro Peer).

7.5 GET /api/sync/errors?limit=N

Liest die letzten N Slots aus dem Error-Ring-Buffer ($N \leq 64$).

7.6 Peer-, WiFi- und Transport-Endpoints

Diese Endpunkte liegen unter `/api/...` (nicht `/api/sync/...`) und kapseln die Peer-, WiFi- und Transport-Features des Agents (§6.5, §6.6).

Methode	Pfad	Funktion
POST	<code>/api/peers/auto_discover</code>	Discovery-Sweep des DHCP-Subnetzes; pflegt <code>peers.json</code> (auto/manual, cluster, self ausgeschlossen)
GET	<code>/api/wifi/scan</code>	SSID-Scan via <code>wpa_supplicant-Control-Socket</code>
POST	<code>/api/wifi/connect</code>	Body {ssid,psk}; setzt ggf. Country (DE) + verbindet wlan0
GET	<code>/api/wifi/status</code>	wlan0-Verbindungsstatus (SSID, Signal, IP)
POST	<code>/api/transport</code>	Body {mode:eth wifi}; exklusiver Transport-Switch; Wi-Fi sticky (Konnektivitäts-Watchdog-Fallback); Wahl persistiert in <code>STATE_DIR/transport</code> (überlebt Reboot)

8. CLI-Tools

8.1 drs_sync_inspect

Pfad: /usr/local/bin/drs_sync_inspect. Read-only-mmap auf /dev/shm/drs-sync.state, formatierte Ausgabe.

```
drs_sync_inspect          # einmaliger Snapshot
drs_sync_inspect -f       # Loop (1 s)
drs_sync_inspect --interval-ms 250 # 4 Hz Refresh
```

Ausgabe:

```
pid=4873 state=LEADER leader=201 term=1
  offset=+0 ns   rate= +0.000 ppm   lat_corr=3962 ns
  pulses=123   holdover_remaining=0 ms   errors=0 (last code 0)
```

8.2 verify_pulse.py

Pfad: tools/verify_pulse.py. Auswertung von Logic-Analyzer-CSV-Exports. Berechnet Δt zwischen GPIO-18-Edges der verschiedenen Pis.

```
./verify_pulse.py capture.csv --threshold-us 100 --out histogram.png
```

Algorithmus:

1. CSV einlesen (Time [s], Channel 0, Channel 1, ...)
2. Pro Kanal rising-edges via Sub-Sample-Linear-Interpolation
3. Pro Kanal-Paar (Referenz=Ch0): nächste Edge im ± 50 ms-Fenster finden
4. Statistik: p50/p99/p99.9/max
5. Exit 0 wenn max < threshold, sonst 1

9. Build-System

Datei: drs_sync/Makefile.

Target	Effekt
make	Baut build/drs_syncd und build/drs_sync_inspect
make test	Baut + führt 5 Unit-Tests aus
make clean	Entfernt build/
make install	Installiert Binaries + systemd-Units (DESTDIR-fähig)

Compiler-Flags:

- -O2 -g -Wall -Wextra -Wpedantic -std=c11
- -Iinclude -Isrc -I../src (für rt_setup/timing aus Parent)
- -D_GNU_SOURCE (recvmsg, SCM_TIMESTAMPING etc.)
- -lpthread -lrt

- Optional `RELEASE=1` → `-O3 -DNDEBUG`

Cross-Compile (vom Dev-Host):

```
CROSS=aarch64-linux-gnu- make
```

Native-Compile (auf Pi): ~5–10 s auf Pi 4B.

Static Asserts:

- `_Static_assert(sizeof(struct drs_packet_v2) == 64)` in `proto_v2.h`
- `#error "DRS sync requires __int128"` in `vclock.c`

Unit-Tests:

Test	Zweck	Resultate
test_crc32	IEEE 802.3 reflected	"123456789" → 0xCB43926 ✓
test_proto	Serialize/Deserialize-Roundtrip	✓
test_min_delay	Outlier-Rejection	✓
test_pi_ctrl	Step + Slew + Rate-Clamp	drift_per_iter ≈ 0 ✓
test_vclock	Rate-Skalierung, Step, Inverse	drift_after_1s=1000007 ns (≈1 ms, 1000 ppm) ✓

10. Deployment

10.1 Voraussetzungen

Dev-Host:

- Linux mit `rsync`, `ssh`, `python3`
- Optional: `avahi-utils` + `libnss-mdns` für mDNS-Auflösung von `.local`
- SSH-Key (`id_ed25519` oder `id_rsa`)

Raspberry Pi 4B:

- Raspberry Pi OS Lite 64-bit (Bookworm oder neuer)
- SSH aktiv, Pubkey hinterlegt
- `gcc` + `make`
- User in `gpio`-Gruppe
- `PREEMPT_RT`-Kernel
- `/boot/firmware/cmdline.txt` mit `isolcpus=3 nohz_full=3 rcu_nocbs=3`

10.2 Erst-Provisionierung

Variante A: Pi schon ge-imaged

```
scp scripts/install-rt-kernel.sh pi@host>:/tmp/
ssh pi@host> 'sudo bash /tmp/install-rt-kernel.sh'
# Pi rebootet am Ende.
```

Das Skript macht: apt-Update, RT-Kernel-Install, cmdline-Append, gpio-Gruppen-Eintrag, /etc/hosts-Eintrag, systemd-Units installieren.

Variante B: SD-Karte komplett frisch flashen

```
sudo ./scripts/flash-sd.sh /dev/sdX [hostname]
# Schreibt Pi-OS-Lite + injiziert SSH-Key + setzt Hostname.
# Danach: Pi booten, dann install-rt-kernel.sh aufrufen.
```

10.3 Code-Update (laufender Pi)

Skript: scripts/deploy-philippknode.sh.

```
# Standard (default HOST=philippknode.lan):
./scripts/deploy-philippknode.sh

# Mit IP statt mDNS-Hostname:
HOST=192.168.1.202 ./scripts/deploy-philippknode.sh

# Anderer User:
REMOTE_USER=myuser HOST=10.0.0.42 ./scripts/deploy-philippknode.sh
```

Workflow:

1. resolve_host() versucht: IPv4-Literal → getent ahostsv4 → avahi-resolve -n4
2. SSH-Connectivity-Check (BatchMode, 5 s Timeout)
3. rsync (mit Excludes für build/, __pycache__, .git, .coverage, report/)
4. Remote-Build via Heredoc:
 - Idempotenter /etc/hosts-Append für eigenen Hostnamen
 - make -j\$(nproc)
 - sudo make install (drs_syncd, drs_sync_inspect, systemd-Units)
 - agent.py + web/* nach /usr/local/share/drs/
 - systemctl daemon-reload + restart für drs-sync und drs-agent
5. Live-Status-Check via drs_sync_inspect

Build-Zeit: ~5–10 s native auf Pi 4B, ~2 s mit -j4.

10.4 Cluster-Deployment

Skript: scripts/deploy-cluster.sh.

```
./scripts/deploy-cluster.sh pi1.lan pi2.lan pi3.lan
DRS_HOSTS="pi1.lan pi2.lan pi3.lan" ./scripts/deploy-cluster.sh
./scripts/deploy-cluster.sh    # liest peers.json
```

11. Hardware-Setup

11.1 Pi 4B-Konfiguration

Kernel-Boot-Parameter (/boot/firmware/cmdline.txt):

```
isolcpus=3 nohz_full=3 rcu_nocbs=3
```

Firmware-Konfiguration (/boot/firmware/config.txt):

```
arm_boost=0
force_turbo=1
arm_freq=1500
```

Deaktiviert DVFS → konstante CPU-Frequenz → keine frequenzabhängige Sync-Drift.

11.2 Netzwerk

Topologie:

- Single-Hop Gigabit-Ethernet
- Layer-2-switched, kein Routing, kein NAT
- Standard MTU 1500
- Recommended: dedicated VLAN für Sync-Traffic

IP-Schema:

- **Primär: routbare DHCP-Adresse.** node-id und IP_MULTICAST_IF nutzen die eth0-/wlan0-IPv4 im Subnetz des Default-Gateways (typ. 192.168.1.x); node-id = letztes Oktett dieser routbaren Adresse (§4.3).
- **Leader = sticky (höchster Term).** Die NodeID ist nur Tiebreak bei gleichem Term (§4.7).
- 10.0.0.XY dient als **DHCP-Down-Fallback** (X=Team-ID 1–8, Y=Node-ID 1–3), Beispiel Team 4 Node 2 → 10.0.0.42.
- **DHCP-Fallback-IPs der Lab-Knoten:** 10.0.0.31 / .32 / .33.

Team-/Node-Zuordnung (6 Teams × 3 Nodes):

Team	Node 1	Node 2	Node 3
1	10.0.0.11	10.0.0.12	10.0.0.13
2	10.0.0.21	10.0.0.22	10.0.0.23
3	10.0.0.31	10.0.0.32	10.0.0.33
4	10.0.0.41	10.0.0.42	10.0.0.43
5	10.0.0.51	10.0.0.52	10.0.0.53
6	10.0.0.61	10.0.0.62	10.0.0.63

Das aktuelle Lab-Cluster läuft auf DHCP 192.168.1.0/24 (philippknode .202 = Leader, drs-node-03 .203 = Follower); die 10.0.0.XY-Tabelle ist die klassenweite statische Konvention und dient als DHCP-Down-Fallback (10.0.0.31/.32/.33).

Pi 4B Transport (philippknode):

- Sync ist interface-agnostisch — *entweder* eth0 *oder* wlan0, nie beide gleichzeitig im selben Subnetz (doppelte IP).
- IP_MULTICAST_IF wird auf die routbare DHCP-Adresse des aktiven Transports gebunden.
- Umschaltung exklusiv via `POST /api/transport {mode:eth|wifi}`; WiFi ist sticky (Konnektivitäts-Watchdog, Fallback nur bei echtem WLAN-Verlust — §6.6, §7).
- Kabelgebunden (eth0): Sub-100- μ s-Sync (NF-1). WiFi (wlan0): ms-Klasse.

11.3 GPIO-Verkabelung für externe Verifikation

```

Pi 4B GPIO-Header (40-pin)
-----
Pin 12 (GPIO 18)  --- Sync-Pulse-Output ----+
Pin 16 (GPIO 23)  --- Health-Output  -----+--- Logic Analyzer
Pin  6 (GND)      --- Common Ground  -----+   (Saleae Logic Pro)

```

Mehrere Pis: Alle GPIO 18 parallel an Analyzer-Kanäle. Alle GNDs verbunden mit Analyzer-GND (kritisch!). 3,3 V Pi-GPIO ist mit den meisten Analyzern direkt kompatibel.

12. Verifikation und Test

12.1 Acceptance-Kriterien (NF-1)

Metrik	Anforderung
$\max(\Delta t)$ über 10 min	$< 100 \mu\text{s}$
$p99(\Delta t)$	$< 80 \mu\text{s}$
$p50(\Delta t)$	$< 50 \mu\text{s}$
GPIO 23 LOW-Dauer	$> 95\%$ der Messdauer
Recovery nach Leader-Crash	$< 11 \text{ s}$ (10 s HOLDOVER + Election)

12.2 Test-Szenarien

Test 1 — Baseline (3 Nodes, 10 min Ethernet): Logic-Analyzer-Capture, `verify_pulse.py`.

Test 2 — Dynamic Discovery: 2 Nodes laufen, konvergieren. Strom für Node 3 ein $\rightarrow$ muss innerhalb 20 s in FOLLOWER konvergieren.

Test 3 — Crash Failure Tolerance: Strom für Leader (lowest ID) ziehen. Verbleibende Nodes: HOLDOVER → Re-Election → neuer Leader innerhalb 11 s. Pulses laufen während Holdover weiter (gefrorene Rate). **Manuell triggerbar via Web-UI "Force Holdover".**

Test 4 — High Jitter Resilience (Out-of-band-Stress): iperf3 saturiert das Management-WLAN, während Sync auf eth0 läuft. Sync auf Ethernet bleibt unbeeinflusst (IRQ-Isolation). *Hinweis:* WiFi ist selbst ein wählbarer Sync-Transport (ms-Klasse, §6.6); dieser Test bezieht sich auf den Fall "Sync kabelgebunden, WLAN als parallele Last".

12.3 Smoketest nach Deploy

```
# 1) Daemon laeuft?
ssh pi@<host> 'sudo systemctl is-active drs-sync.service'

# 2) State + Pulses?
ssh pi@<host> 'sudo /usr/local/bin/drs_sync_inspect'

# 3) GPIO-Pegel?
ssh pi@<host> 'gpioget --chip gpiochip0 18 23'

# 4) HTTP API?
curl -s http://<host>:8080/api/sync/state | python3 -m json.tool
curl -s http://<host>:8080/api/sync/health | python3 -m json.tool

# 5) Web-UI?
xdg-open http://<host>:8080
```

12.4 Aktuelle Verifikationsstatus

Multi-Node verifiziert (2026-05-25, philippknode + drs-node-03):

- Leader/Follower-Election stabil (LEADER 201, FOLLOWER, term konsistent), Errors = 0
- Regler rate rastet bei ~ -1 ppm ein (weit innerhalb des -1000 -ppm-Clamps); Telemetrie-offset $< 100 \mu\text{s}$ stabil
- Externe GPIO-Cross-Link-Messung (tools/gpio_xprobe.c): Inter-Node-Pulsversatz **p50 17 μs , max 26 μs , Drift 0.055 ppm** über 90 s → Regressions-Gate **PASS**; Follower-Pulsperiode **1000.0005 ms** (kabelgebundener Skew)
- NF-1 ($< 100 \mu\text{s}$) erfüllt auf dem Stock-Pi-4B (Software-Timestamping), **nur kabelgebunden**
- lat_corr $\approx 3900\text{--}4100$ ns (stabil über mehrere Restarts)

WLAN-Sync verifiziert:

- WiFi als Sync-Transport funktionsfähig (ms-Klasse, **nicht** NF-1)
- WiFi-Follower-Offset $\sim 2,6 \mu\text{s}$ bei ruhigem AP (Telemetrie)
- WiFi-RTT-Mittel $\sim 0,87$ ms (Spanne $0,70 / 1,63$ ms) ggü. kabelgebunden $\sim 0,15$ ms — Faktor ~ 6 langsamer
- Constraint bestätigt: WiFi kann **nicht** mit eth0 im selben Subnetz koexistieren (doppelte IP) → exklusiver Transport-Switch (§6.6); der AP muss **Client-Isolation deaktivieren** (WiFi↔LAN brücken)

Failover verifiziert:

- Nach Leader-Disconnect läuft der pending SYNC_REQ ab / wird gelöscht; der Follower re-acquired sauber auf den neuen Leader (Discipline-Reset via `locked_leader_id`, §4.8)
- Sticky Leadership bestätigt: ein zurückkehrender Knoten (Neustart/Term 0) wird FOLLOWER, der amtierende Leader behält die Führung (§4.7) — kein Leader-Flap bei Node-Restart

Noch ausstehend:

- Logic-Analyzer-Capture als zweiter, unabhängiger NF-1-Beleg
- Test-Szenarien 2–3 (Late-Join, Leader-Crash) mit 3 Knoten

13. Troubleshooting & FAQ

13.1 SSH-Probleme

Symptom: `Permission denied (publickey)`

- Prüfen: User `pi` (Default `flash-sd.sh`) oder Custom-User?
- `ssh -v pi@<host>` zeigt welcher Key geboten wird
- Lösung: `ssh-copy-id pi@<host>` oder SD-Karte mounten und Pubkey in `authorized_keys` schreiben

Symptom: `<host>` kann nicht aufgelöst werden

- `.lan`-TLDs sind keine mDNS → nicht via `avahi` auflösbar
- Workarounds: IP direkt, `/etc/hosts`, oder `avahi` installieren (nur `.local`)

13.2 Daemon startet, aber `pulse_count` bleibt 0

- Hat der Pi `PREEMPT_RT`? `uname -a | grep PREEMPT_RT`
- Sind die Cmdline-Parameter aktiv? `cat /proc/cmdline`
- Ist gpio-Gruppen-Mitgliedschaft aktiv? `groups pi`
- Pulse-Engine braucht ~3s nach Daemon-Start (GROUND 2s + CALIBRATION + LISTEN)
- **Follower mit `lat_corr=0` und `pulses=0`:** prüfe, ob der Knoten GROUND/CALIBRATION/LISTEN durchlaufen hat — der `sm_is_active`-Guard (§4.8) stellt sicher, dass ein Knoten erst nach Abschluss von CALIBRATION/LISTEN FOLLOWER wird (vclock und Pulse-Engine initialisiert)

13.3 `lat_corr` ist absurd groß (>1s)

- Ursache: ein Mix der Clock-Domains (`CLOCK_REALTIME` aus einem `SO_TIMESTAMPING`-msg, gemischt mit `CLOCK_MONOTONIC_RAW`) erzeugt 10^{18} -Werte. Der Daemon nutzt daher durchgängig Userspace-Timestamping in einer einzigen Clock-Domain
- Prüfen, ob `recv_with_ts` in `calibrate.c` und `net_io.c` nur `ts_now_ns()` nutzt
- Healthy-Wert auf Pi 4B: 3000–5000 ns

13.4 sudo: unable to resolve host

- `/etc/hosts` auf dem Pi hat keinen Eintrag für `\$(hostname)`
- `deploy-philippknode.sh` ergänzt das idempotent
- Manuell: `echo "127.0.1.1 \$(hostname)" | sudo tee -a /etc/hosts`

13.5 State bleibt CANDIDATE / kommt nie zu LEADER

- Single-Node: Election-Timeout (250–500 ms) muss durchgelaufen sein → max 1 s warten
- Multi-Node: existiert bereits ein amtierender Leader (höherer Term)? Dann wird der Knoten korrekt FOLLOWER statt LEADER (sticky Leadership, §4.7). Bei echter gleichzeitiger Election entscheidet die niedrigere NodeID (Tiebreak) → ggf. LOCK-Mode prüfen
- `sudo journalctl -u drs-sync -f` zeigt Transitionen

13.6 Web-UI: Cluster-Tabelle zeigt "unreachable"

- Peer-Agent läuft nicht / Port 8080 nicht erreichbar
- `peers.json` prüfen: `agent_port` korrekt? IP erreichbar?
- Curl-Test: `curl http://<peer-ip>:8080/api/sync/state`

14. Performance-Budget

Ziel (NF-1): $\max(\Delta t_{\text{physical}}) < 100 \mu\text{s}$ zwischen Pi-Knoten.

Komponente	Budget	Realität Pi 4B	Maßnahme
NIC-Timestamp-Jitter	$< 20 \mu\text{s}$	$\sim 5\text{--}10 \mu\text{s}$ (Userspace-TS)	Min-Delay-Filter
Scheduler-Latency	$< 30 \mu\text{s}$	$< 5 \mu\text{s}$ (PREEMPT_RT)	SCHED_FIFO 85 + isolc- pus
Filter-Residual	$< 20 \mu\text{s}$	$< 10 \mu\text{s}$	Window=10, min-select
GPIO-Generation	$< 20 \mu\text{s}$	$< 1 \mu\text{s}$ (/dev/gpiomem)	Register-Direkt
Total worst	$< 100 \mu\text{s}$	erfüllt: p50 17 μs / max 26 μs (GPIO-Cross-Link)	Closed-Loop PI-Regler

Mess-Realität:

- `lat_corr` (Loopback-Latenz) = 3900–4000 ns ($\approx 4 \mu\text{s}$)
- Round-Trip auf Loopback: $\sim 8 \mu\text{s}$
- Realer Network-RTT erwartet: 50–150 μs auf 1G-Switch

15. Erweiterungs-Guide

15.1 Warum single-threaded?

Initial-Plan: separater Pulse-Thread auf Core 3. Verworfen weil:

- Zwei Threads auf demselben isolierten Core erzeugen Context-Switches
- Race-Conditions zwischen Pulse-Timer und State-Machine bei Rate-Updates

Aktuell: **ein** RT-Thread mit `epoll_wait`, der Network-Events UND Timer gleichzeitig verarbeitet. Sequentialisierung garantiert Konsistenz ohne Locks.

15.2 Wie erweitern ohne Hot-Path zu stören?

Erlaubt:

- Neue Telemetrie-Felder in `struct drs_telemetry` (atomare Writes)
- Neue Command-Codes via `cmd.h`-Enum
- Neue API-Endpoints im Python-Agent (Cores 0-2)
- Logging im Agent (außer dem Hot-Path)

Verboten:

- `printf` / `fprintf` / `syslog` im Sync-Thread (NF-9)
- `malloc` / `realloc` / `free` nach `mlockall` (NF-12)
- Mutex-Locks im Sync-Loop (NF-15)
- Floating-Point-Operationen im Sync-Thread
- Disk-I/O (auch indirekt via Library-Calls)

15.3 Neuen Test hinzufügen

1. `tests/test_<modul>.c` schreiben (assert-basiert, return 0 on success)
2. In Makefile als `TEST_BINS` eintragen
3. `make test` → läuft automatisch

15.4 Neuen API-Endpoint hinzufügen

1. Read-Helper in `agent/agent.py`
2. In `_dispatch_api`-Kette einen neuen if-Block
3. `make install + systemctl restart drs-agent` → live

16. Glossar

Term	Bedeutung
DRS	Distributed Real-Time System
Sticky Leadership	Term-basierte Election: höchster Election-Term gewinnt, amtierender Leader bleibt bis zum Ausfall; NodeID nur Tiebreak bei gleichem Term
Election-Term	Monoton steigende Runden-Nummer; maßgeblich für Leadership (höchster Term gewinnt)
Cmdline	Linux-Kernel-Boot-Parameter (/proc/cmdline)
HOLDOVER	Freerun-Modus mit eingefrorener Rate, kein Sync-Update
Lock-Mode	Manueller Override gegen die automatische Election (AUTO/-LOCK_AS_LEADER/LOCK_AS_FOLLOWER)
mDNS	Multicast-DNS, Auflösung von .local-Hostnamen via avahi
PHC	PTP Hardware Clock (Pi 4B BCM54213PE hat keinen)
PHY	Physical Layer (Ethernet-Chip)
PREEMPT_RT	Linux-Real-Time-Kernel-Patch, deterministische Latenzen
PTP	Precision Time Protocol (IEEE 1588), Sub- μ s-Sync mit HW-Support
Q32.32	Festkomma: 32 Bit Integer + 32 Bit Fraction (signed 64-bit)
Q16.16	Festkomma: 16 Bit Integer + 16 Bit Fraction (signed 32-bit)
Seqlock	Lock-frei-Konsistenz via geraden/ungeraden Sequence-Counter
Step-Mode	Hard-Phase-Sprung der virtuellen Uhr bei $ \theta > 1$ ms
Slew-Mode	Sanfte Frequenz-Korrektur via PI-Regler bei $ \theta \leq 1$ ms
SCHED_FIFO	Real-Time-Scheduler, läuft bis es selbst blockt
isolcpus=N	Kernel-Parameter: CPU N wird nicht vom Scheduler bedient
nohz_full=N	Kernel-Parameter: kein Scheduler-Tick auf CPU N
rcu_nocbs=N	Kernel-Parameter: keine RCU-Callbacks auf CPU N
T1/T2/T3/T4	PTP-Zwei-Wege-Timestamps
θ (theta)	Phase-Offset zwischen Follower-Uhr und Leader-Uhr
δ (delta)	Round-Trip-Delay (Pure-Network-RTT)
ppm	parts-per-million, Frequenz-Abweichung (10^{-6})

17. Anhang: Dateipfade und Code-Referenzen

17.1 Verzeichnisstruktur

```

rpi_zeitsynchronisation/
|-- src/                                # Legacy RTT-Messsystem
|   |-- drs_master.c                    # RTT-Client
|   |-- drs_echo.c                      # UDP-Echo-Daemon
|   |-- rt_setup.{c,h}                  # mlockall/CPU-Pin/SCHED_FIFO
|   |-- timing.h                        # ns-Helfer
|   `-- ...
|-- drs_sync/                           # DIESES Subprojekt
|   |-- src/
|   |   |-- drs_syncd.c                 # Main-Loop, ~370 LOC
|   |   |-- proto_v2.{c,h}              # 64-byte DRS-T-Wire-Protokoll
|   |   |-- crc32.{c,h}                 # IEEE 802.3 reflected
|   |   |-- vclock.{c,h}                # Virtuelle Uhr (Seqlock + Q32.32)
|   |   |-- statemachine.{c,h}          # 7-State FSM + Lock-Mode
|   |   |-- election.{c,h}              # Sticky term-basierte Leadership
|   |   |-- pi_ctrl.{c,h}               # Dual-Loop PI (Step + Slew)
|   |   |-- min_delay.{c,h}             # Rolling-Window Outlier-Filter
|   |   |-- calibrate.{c,h}             # Loopback-Latenz
|   |   |-- net_io.{c,h}                # UDP-Sockets + Multicast
|   |   |-- gpio_bcm2711.{c,h}         # /dev/gpiomem mmap

```

```

| | |-- pulse_engine.{c,h}    # GPIO 18/23
| | |-- telemetry.{c,h}      # /dev/shm/drs-sync.state
| | |-- errors.{c,h}         # Ring-Buffer + Codes
| | |-- cmd.{c,h}            # /dev/shm/drs-sync.cmd
| |-- include/drs_sync_config.h
| |-- tests/                 # 5 C-Unit-Tests + pytest
| |-- tools/
| | |-- drs_sync_inspect.c   # CLI-Telemetrie-Reader
| | |-- verify_pulse.py      # Logic-Analyzer-Auswertung
| |-- systemd/               # 3 Units
| |-- docs/                  # Diese Dokumentation + Sub-Docs
| |-- Makefile
|-- agent/agent.py           # Python HTTP API + UI (~1700 LOC)
|-- web/                     # 5-Tab SPA
|-- scripts/
| |-- flash-sd.sh
| |-- install-rt-kernel.sh
| |-- deploy-philippknode.sh
| |-- deploy-cluster.sh
|-- systemd/                 # Legacy-Units

```

17.2 Wichtige Konstanten

Alle in include/drs_sync_config.h:

```

DRS_MAGIC_V2          = 0x44525354
DRS_VERSION           = 0x02
DRS_PAYLOAD_BYTES     = 64
DRS_PORT              = 47200
DRS_MCAST_GROUP       = "239.192.88.100"
DRS_CALIB_LOOPBACK_PORT = 47201

DRS_GROUND_DURATION_NS = 2 s
DRS_LEADER_ANNOUNCE_NS = 100 ms
DRS_SYNC_EXCHANGE_NS   = 50 ms
DRS_SYNC_REQ_TIMEOUT_NS = pending-SYNC_REQ-Ablauf (Failover-Re-Acquire, section 4.8)
DRS_FOLLOWER_TIMEOUT_NS = 300 ms (3 x ANNOUNCE)
DRS_HOLDOVER_MAX_NS    = 10 s
DRS_ELECTION_MIN_NS    = 250 ms
DRS_ELECTION_MAX_NS    = 500 ms
DRS_WIFI_WATCHDOG_S    = 15 s (connectivity-watchdog interval, section 6.6)
DRS_WIFI_FAIL_LIMIT    = 3 (consecutive failures before WiFi->eth0 fallback, section 6.6)

DRS_PULSE_PERIOD_NS    = 1 s
DRS_PULSE_HIGH_DURATION_NS = 10 ms

DRS_STABLE_WINDOW_NS   = 10 s
DRS_STABLE_OFFSET_THRESH_NS = 100 us

DRS_MIN_DELAY_WINDOW   = 10
DRS_MIN_DELAY_TOLERANCE_NS = 10 us

DRS_CALIB_SAMPLES      = 50
DRS_CALIB_OUTLIER_NS   = 20 us

DRS_PI_KP_Q16          = 3276 (~ 0.05)
DRS_PI_KI_Q16          = 327 (~ 0.005)
DRS_PI_UPDATE_PERIOD_NS = 50 ms

DRS_STEP_THRESHOLD_NS  = 1 ms
DRS_STEP_CONFIRM_SAMPLES = 3

DRS_RATE_NOMINAL_Q32   = 0x100000000 (1.0)

```

DRS_RATE_PPM_Q32	= 4295	(~ $2^{32}/1e6$)
DRS_RATE_MAX_PPM	= 1000	
DRS_RT_CPU	= 3	
DRS_RT_PRIO	= 85	
DRS_GPIO_PULSE_PIN	= 18	
DRS_GPIO_HEALTH_PIN	= 23	

17.3 Sub-Dokumentationen

- docs/PROTOCOL_V2.md — Wire-Format-Spezifikation
- docs/STATE_MACHINE.md — Transitions-Tabelle + Filter-Reset-Trigger
- docs/ERRORS.md — Error-Code-Tabelle + Probe-Effekt-Disziplin
- docs/VERIFICATION.md — Externe Falsifizierbarkeit

Ende der Dokumentation.